Thisolver

CI failure triage

Sort the red build. Fix only what is real. Check the fix.

Jev for judgement · deterministic code for policy · IBM Bob for the fix

The problem

Most of the time goes on sorting, not fixing

- CI goes red. A developer opens four jobs of logs.
- Is it flaky? Infra? A dependency that moved? Or did I break something?
- That triage is the expensive part. The fix is often one line.
- So teams learn to ignore red builds — and the one real regression hides in the noise.

```
× test (3.12)
× integration
× deps (windows-latest)
× release-notes

4 failing jobs.
How many actual problems?
```

The pipeline

Each stage does the job it is actually suited to



Splitting a log is a **parsing problem**.
So code does it.

What a failure *is* is **judgement under uncertainty**.
So Jev does it.

What to *do* is **policy**.
So readable code does it.

Split

One unit per failing test or step — no model involved

- pytest, jest/vitest, go test, plus a step-level fallback
- Timestamps and ANSI colour stripped
- Signatures are *scored*, not first-matched
- The same failure across six matrix legs collapses into one unit

Six Python versions failing one lint hook is one bug — reporting it six times would spend six agents on a single fix.

```
# pydantic/pydantic, a real run
before: 7 units
after:  2 units

config   64% human x6 jobs
   Lint..........Failed
flaky    46% human
```

Triage

One state, three typed questions, calibrated probabilities

```
POST /v1/systemone

category    Choice
  regression | flaky | infra
  | dependency | config

caused_by_pr Noul
  did this PR's files cause it?

severity    Score
  cosmetic .. release blocker
```

- The state is the log excerpt, the files the traceback named, and the PR diff summary
- Jev returns probabilities, not prose — so code can branch on them
- Attribution is measured over *source* files, not the test file

A failing test always names its own file, so counting it would dilute the signal identically on every unit.

Route

Confidence gates automation

CATEGORY	CONFIDENCE	ACTION
any	$< 65\%$	human -- nothing automated on a guess
flaky	$\geq 65\%$	rerun once, label, quarantine
infra	$\geq 65\%$	ping platform -- no code change helps
dependency	$\geq 65\%$	ping owner -- pinning is a human call
config	$\geq 65\%$	human -- fix lives outside the code
regression	$\geq 75\%$ and from this PR	dispatch to Bob

Anything TriSolver is unsure about goes where it would have gone anyway: to a person.

Dispatch

One self-contained brief per regression

- The failure, the implicated files, the PR diff, and the repo's own CONTRIBUTING / runbook
- Acceptance criteria stated up front — word for word the gates the patch will face
- Three adapters: `cli` (headless `bob run`), `inbox`, `fixture`
- Units are independent, so regressions fan out one per subagent

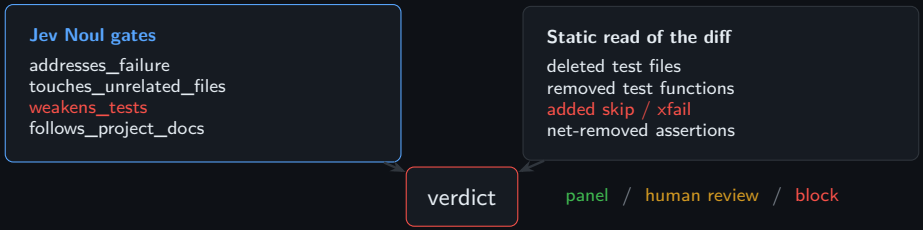
```
## Acceptance criteria

Rejected if it:
1. does not address the failure
2. touches unrelated files
3. deletes, skips or weakens a test
4. contradicts the conventions

Fix the code. Do not fix the test.
```

Verify

Two independent checks — either one blocks alone



A novel cheat the regex misses is still caught by the model. A model having a bad day still cannot wave through a deleted test.

The demo moment

The agent deleted the failing test

```
BLOCK     test_discount_applied

  addresses_failure      0.15  ###.................
  touches_unrelated_files 0.01  ....................
  weakens_tests          0.95  ###################.
  follows_project_docs    0.25  #####...............

  static check    removed test(s): test_discount_applied
  files touched   tests/test_pricing.py

  BLOCKED by static check  BLOCKED by Jev
  The failing test was neutralised rather than the defect fixed.
```

Deleting the failing test is the well-known way to make CI green. It is caught automatically, twice, by two independent mechanisms.

A real run

Self-hosted runner, one build, five kinds of failure

```
CATEGORY     CONF   PR  SEVERITY      ACTION
-------------------------------------
regression    97%  94%  major       -> bob       test_discount_applied
   AssertionError: assert 4977 == 4978
dependency    97%  12%  major         ping deps
config        97%  12%  minor         human
infra         97%  12%  minor         ping infra
flaky         51%  12%  minor         human  (below floor)

5 failure unit(s): 1 -> bob, 2 human, 1 ping deps, 1 ping infra
```

usernotfound-101/testrepobob — a real PR, real GitHub Actions logs, four jobs red. Four routed away from Bob.

Bob fixes the one that was real

And the gates check it before a human sees it

```
bob run -mode agent
task 3ed0eeca...  149s  $0.40
13 tool calls

-- a/src/storefront/pricing.py
+++ b/src/storefront/pricing.py
-    return int(total)
+    return round(total)
```

```
SURFACE   test_discount_applied

addresses_failure      0.88
touches_unrelated_files 0.01
weakens_tests          0.04
follows_project_docs   0.82

static: no test weakening
files: src/storefront/pricing.py
```

CONTRIBUTING.md says money is integer cents and rounding happens once, at the end. Bob's fix matches that rule — and the test suite passes.

What is proven, and what is not

Validated

- Live ingestion against real failed runs in `psf/requests`, `pydantic/pydantic`, `urllib3/urllib3`, `encode/httpx`
- A purpose-built sandbox on a self-hosted runner: five categories, one build
- Bob's real patch cleared all four gates; the suite passes
- 75 tests

Not yet

- Jev runs in a labelled offline simulator — no API key yet. Thresholds are tuned for real calibrated probabilities
- That is why the flaky unit scored 51% and fell to a human
- Subagent fan-out is built but undemonstrated: this run had one regression
- No watsonx.ai or watsonx Orchestrate

Sort the red build.
Fix only what is real.
Check the fix.

TriSolver · Jev for judgement · IBM Bob for the fix